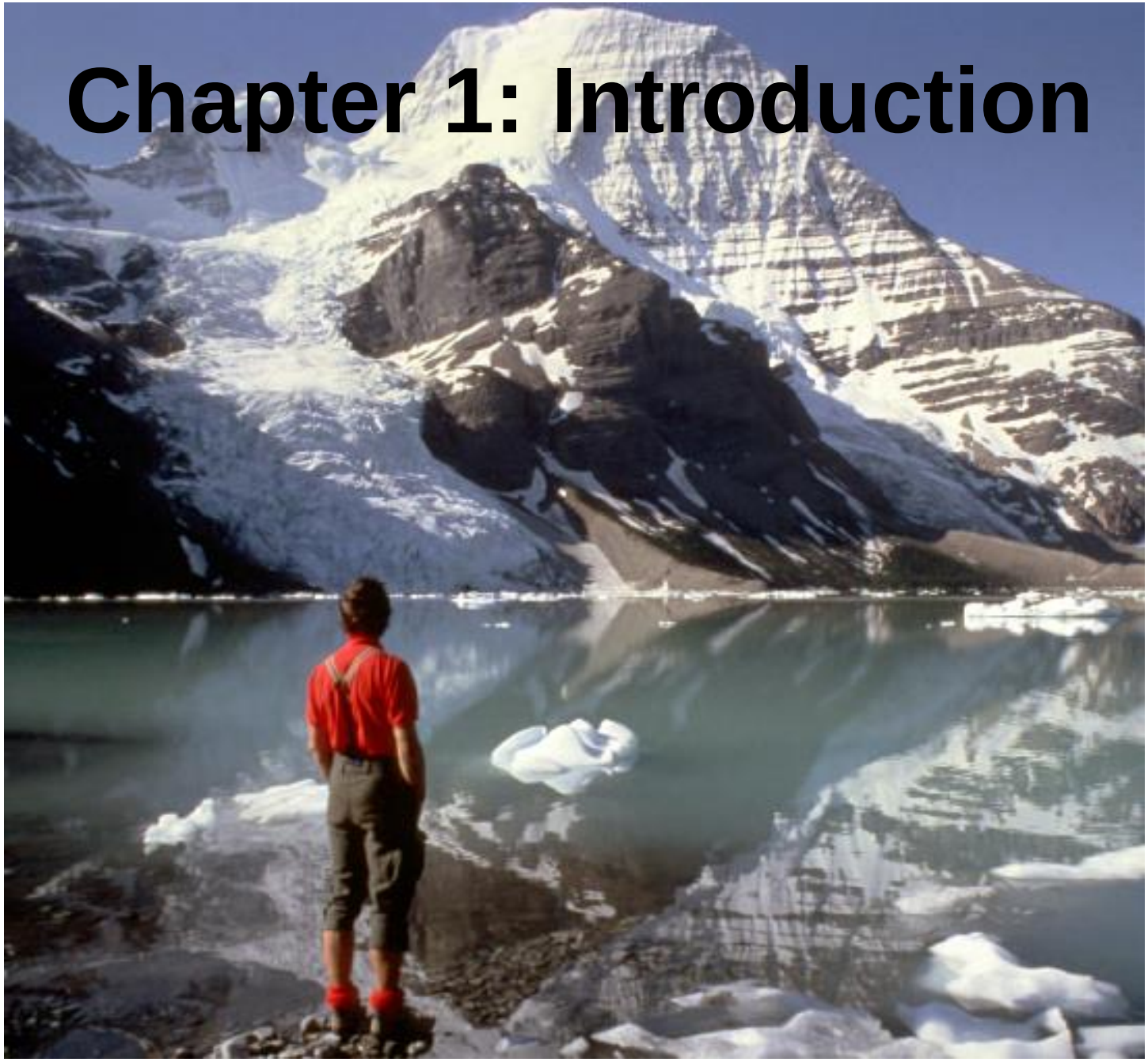
Object-Oriented Software Engineering
Using UML, Patterns, and Java

Chapter 1: Introduction



Objectives of the Lectures

- Appreciate the Fundamentals of Software Engineering:
  - Methodologies
  - Process models
  - Description and modeling techniques
  - System analysis - Requirements engineering
  - System design
  - Implementation: Principles of system development

Assumptions for this Class

- Assumption:
  - You have taken CS 2511(Programing 2), CS3611 (Visual Basic & DB) or CS3810 (DS & Algorithms)
- Beneficial:
  - You have had practical experience with a large software system
  - You have already participated in a large software project
  - You have experienced major problems.

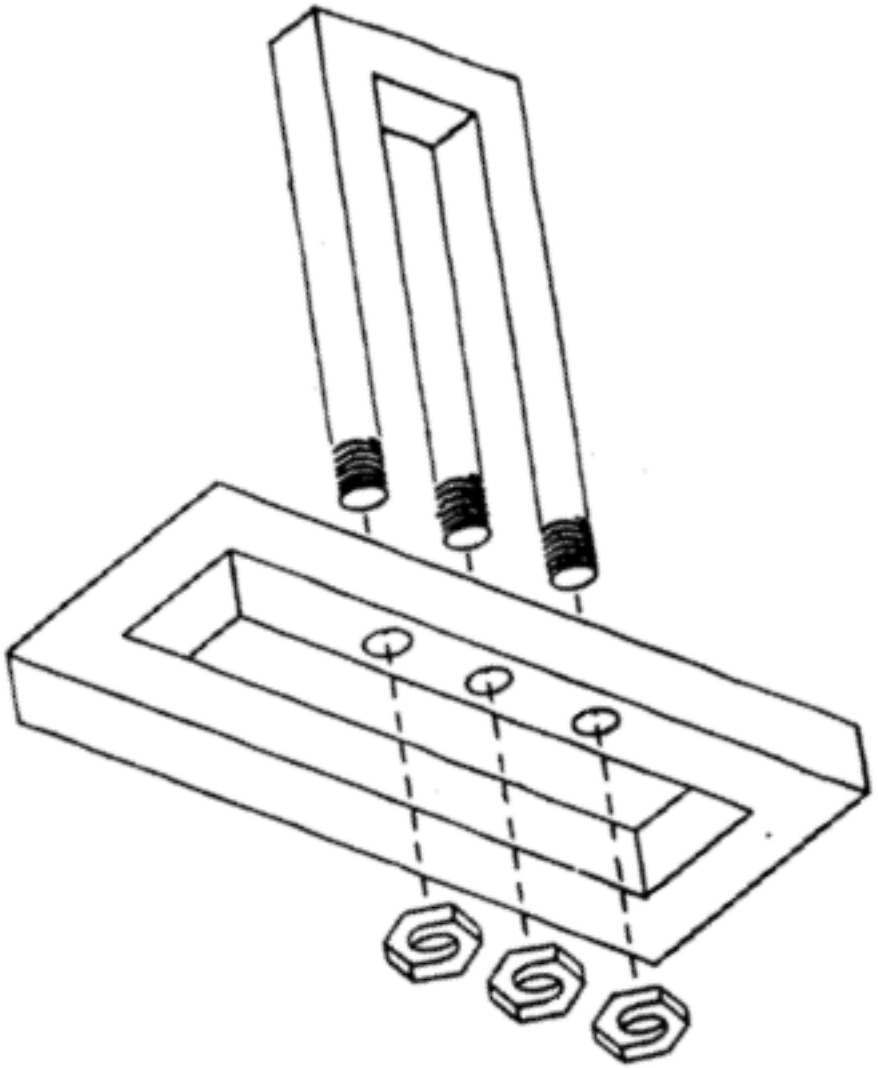
Focus: Acquire Technical Knowledge

- Different methodologies ("philosophies") to model and develop software systems
- Different modeling notations
- Different modeling methods
- Different software lifecycle models (empirical control models, defined control models)
- Different testing techniques (eg. vertical testing, horizontal testing)
- Rationale Management
- Release and Configuration Management

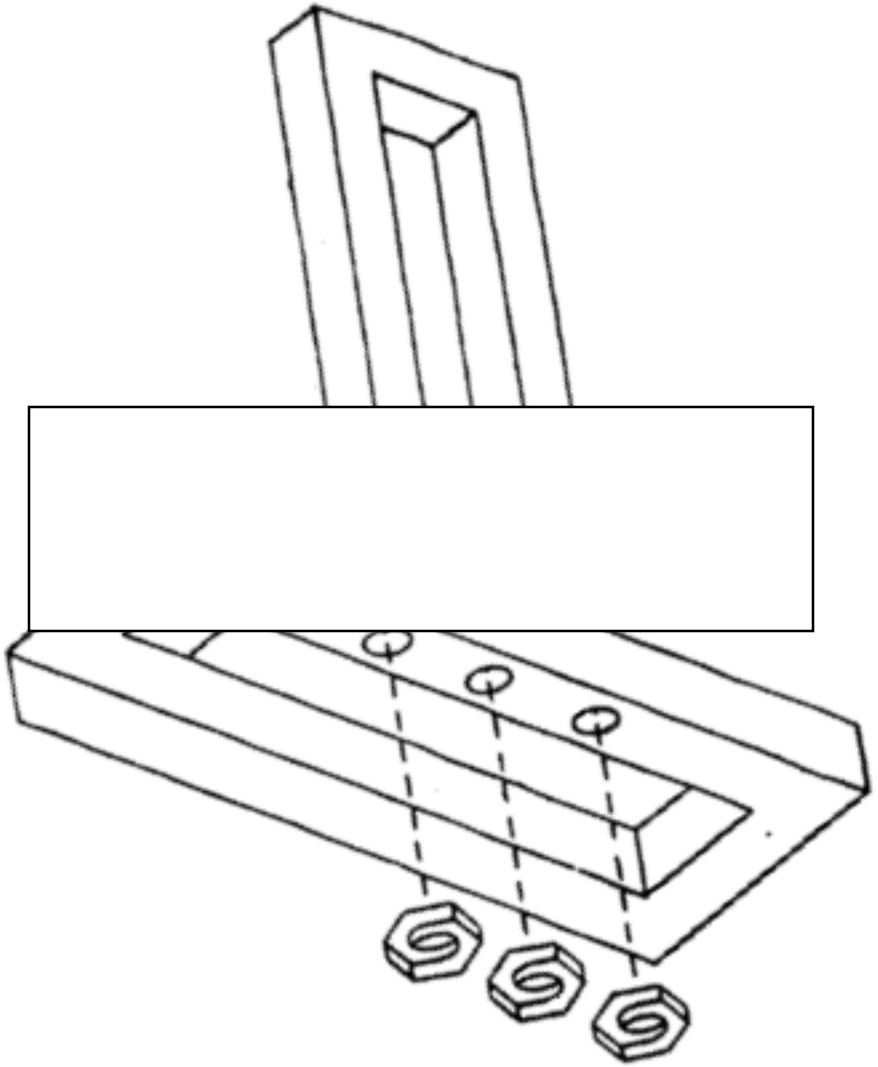
Outline of Today's Lecture

- The development challenge
- Dealing with change
- Concepts: Abstraction, Modeling, Hierarchy
- Methodologies

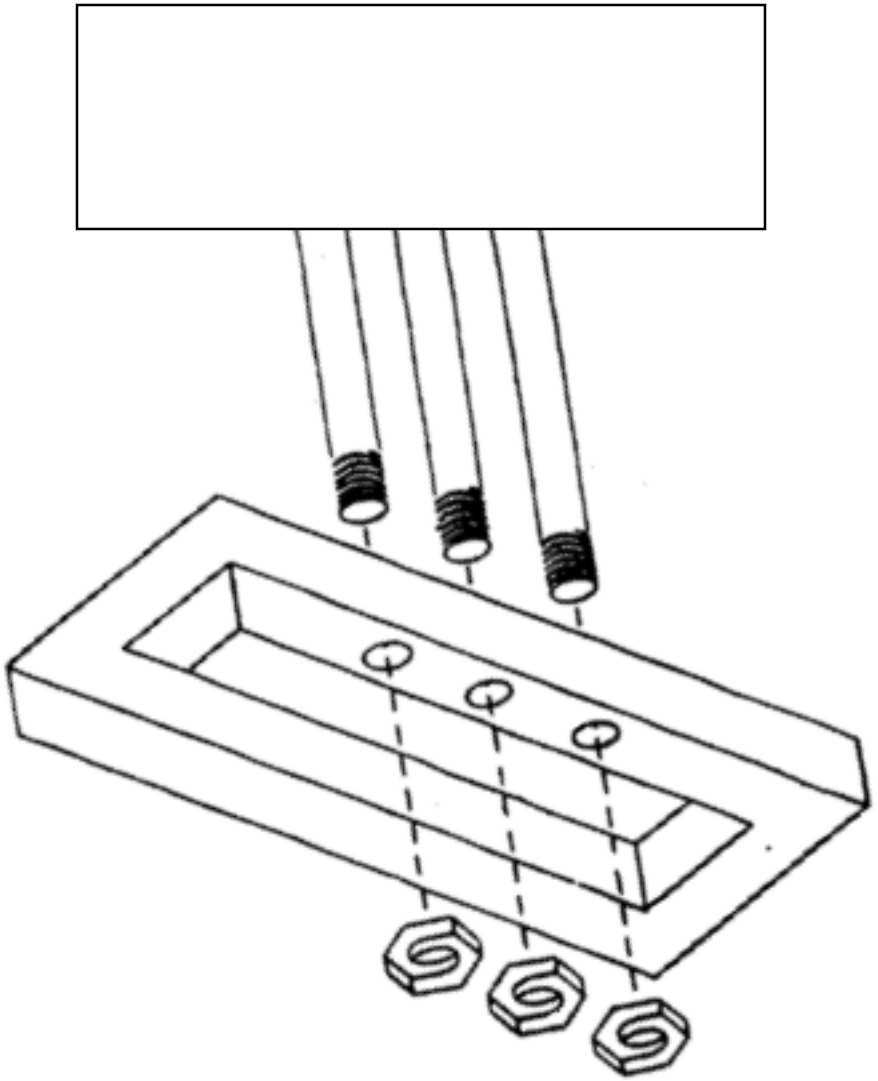
Can you develop this system?



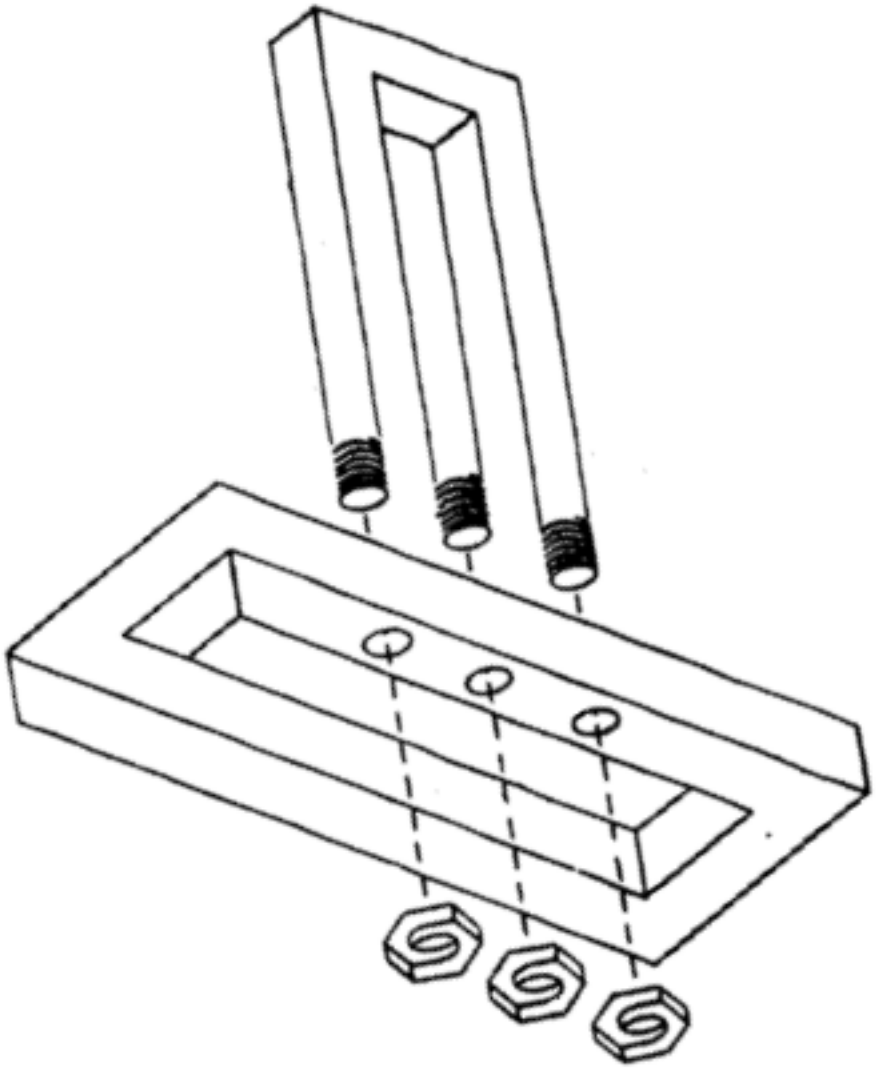
Can you develop this system?



Can you develop this system?



Can you develop this system?

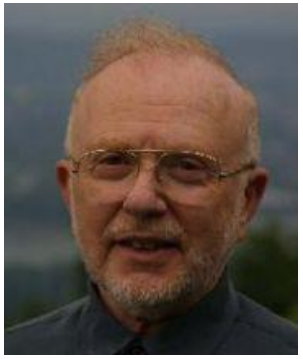


The impossible Fork

Why is Software Development difficult?

- The problem is usually ambiguous
- The requirements are usually unclear and changing when they become clearer
- The problem domain (called application domain) is complex, and so is the solution domain
- The development process is difficult to manage
- Software offers extreme flexibility
- Software is a discrete system
  - Continuous systems have no hidden surprises
  - Discrete systems can have hidden surprises!

David Lorge Parnas - an early pioneer in software engineering who developed the concepts of modularity and information hiding in systems which are the foundation of object oriented methodologies.



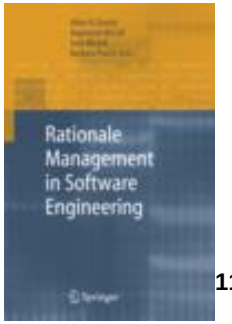
Software Development is more than just Writing Code



- It is problem solving
  - Understanding a problem
  - Proposing a solution and plan
  - Engineering a system based on the proposed solution using a ***good*** design
- It is about dealing with complexity
  - Creating abstractions and models
  - Notations for abstractions
- It is knowledge management
  - Elicitation, analysis, design, validation of the system and the solution process
- It is rationale management
  - Making the design and development decisions explicit to all stakeholders involved.







Can we not use the Scientific Method?

- Not exactly, we need ideas and hypotheses
  - The scientific method, unfortunately, has never quite gotten around to saying exactly where to pick up these hypotheses.
- The traditional scientific method has always been at the very best, 20-20 hindsight
  - It's good for seeing where you've been. It's good for testing of what you think you know
  - But it can't tell you where you should to go
- Creativity, originality, inventiveness, intuition, imagination – "unstuckness," in other words – are completely outside the domain of the scientific method
  - Robert Pirsig, Zen and the Art of Motorcycle Maintenance, p. 251, Bantam Books, 1984.

Techniques, Methodologies and Tools

- **Techniques:**
  - Formal procedures for producing results using some well-defined notation
- **Methodologies:**
  - Collection of techniques applied across software development and unified by a philosophical approach
- **Tools:**
  - Instruments or automated systems to accomplish a technique

Computer Science vs. Engineering

- Computer Scientist
  - Assumes techniques and tools have to be developed.
  - Proves theorems about algorithms, designs languages, defines knowledge representation schemes
  - Has infinite time…
- Engineer
  - Develops a solution for a problem formulated by a client
  - Uses computers & languages, techniques and tools
- Software Engineer
  - Works in multiple application domains
  - Has only 3 months...
  - …while changes occurs in the problem formulation (requirements) and also in the available technology.

Software Engineering: A Working Definition

Software Engineering is a collection of techniques, methodologies and tools that help with the production of

A high quality ***software*** system developed with a given *budget* before a given *deadline* while *change* occurs

Challenge: Dealing with complexity and change

Software Engineering: A Problem Solving Activity

- **Analysis:**
  - Understand the nature of the problem and break the problem into pieces
- **Synthesis:**
  - Put the pieces together into a large structure

For problem solving we use techniques, methodologies and tools.

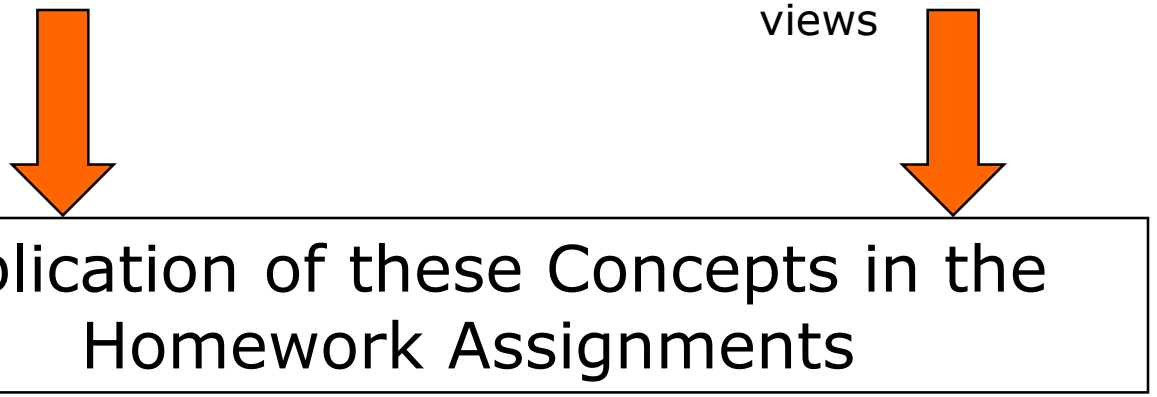
Course Outline

Dealing with Complexity

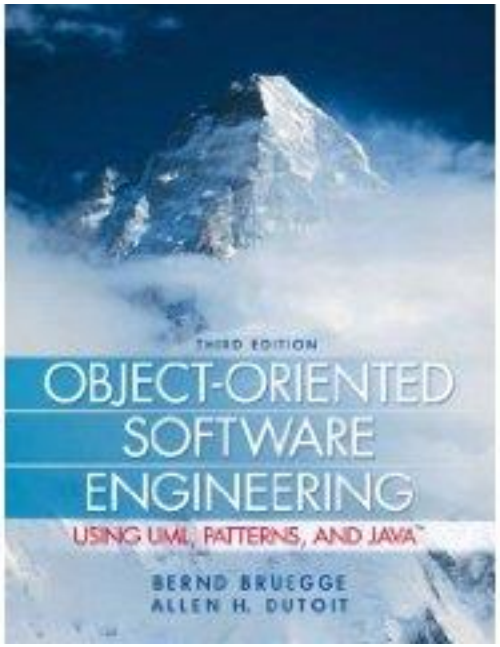
- Notations (UML, OCL)
- Requirements Engineering, Analysis and Design
  - OOSE, SA/SD, scenario-based design, formal specifications
- Testing
  - Vertical and horizontal testing

Dealing with Change

- Rationale Management
  - Knowledge Management
  - Patterns
- Release Management
  - Configuration Management, Continuous Integration
- Software Life Cycle
  - Linear models
  - Iterative models
  - Activity-vs Entity-based views



Textbook



Bernd Bruegge, Allen H. Dutoit

Object-Oriented Software Engineering: Using UML, Patterns and Java, 3rd Edition

Publisher: **Prentice Hall**, Upper Saddle River, NJ, 2009;

ISBN-10: 0136061257

ISBN-13: 978-0136061250

- Additional readings will be added during each lecture.

What to do next?

- Reading for the next two weeks
  - Chapter 1 and 2, Bruegge&Dutoit, Object-Oriented Software Engineering

Additional Slides

